Machine Learning Practice With Python

ITER, SOA University

Centre for Data Science
SOA University

Contents

CSE 3255	Machine Learning Practice with Python	3	2
Introduction, Building a Movie Recommendation Engine with Naïve Bayes, Predicting Online Ad Click-Through with Tree-Based Algorithms, Predicting Online Ad Click-Through with Logistic Regression, Predicting Stock Prices with Regression Algorithms, Predicting Stock Prices with Artificial Neural Networks, Mining the 20 Newsgroups Dataset with Text Analysis Techniques, Recognizing Faces with Support Vector Machine.		Textbook - Python Machine Learning By Example by Yuxi Liu, Packt Publishing Weekly Course Format: 2L - 2P	

Table: Marks Distribution

Component	Marks
Mid-sem(Internal)	15
Assignment	10
Quiz	10
Attendance	05
Lab exam	20
End-Sem(Final)	40

What is Machine Learning?

- A term coined around **1960**, combining "machine" (computer, robot, device) and "learning" (acquiring or discovering event patterns).
- Mimics human thinking by interacting with input data, expected output, and the environment to derive patterns represented by mathematical models.
- Unlike traditional programming, it does **not receive explicit and instructive coding**.
- **Examples**: facial recognition, language translation, responding to emails, making data-driven business decisions, and creating various types of content.

Why We Need Machine Learning

- **Automation of Routine Tasks**: Machines are better suited for routine, repetitive, or tedious tasks, mitigating risks from human fatigue or inattention.

Figure: An example of a self-driving car

- **Superior Performance**: Machines can perform comparably or even better than human domain experts by utilizing collective intelligence (e.g., diagnosing certain types of cancer, AlphaGo beating humans).

Why We Need Machine Learning

- **Human-Machine Synergy**: Leads to the most intelligent and tireless systems.

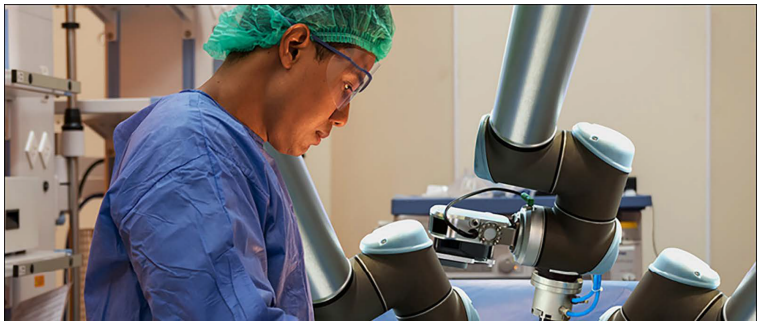


Figure: AI-assisted surgery

- **AI-Generated Content (AIGC)**: A recent breakthrough using AI technologies to create or assist in creating content like articles, music, images, and videos.
- **Business & Individual Benefits**: Enables businesses to understand customer behavior and optimize operations; enhances daily life with applications like spam email filtering and online advertising.

ML vs. Traditional Programming

- A popular myth suggests machine learning is the same as automation because it performs instructive and repetitive tasks.
- **The Core Difference**:

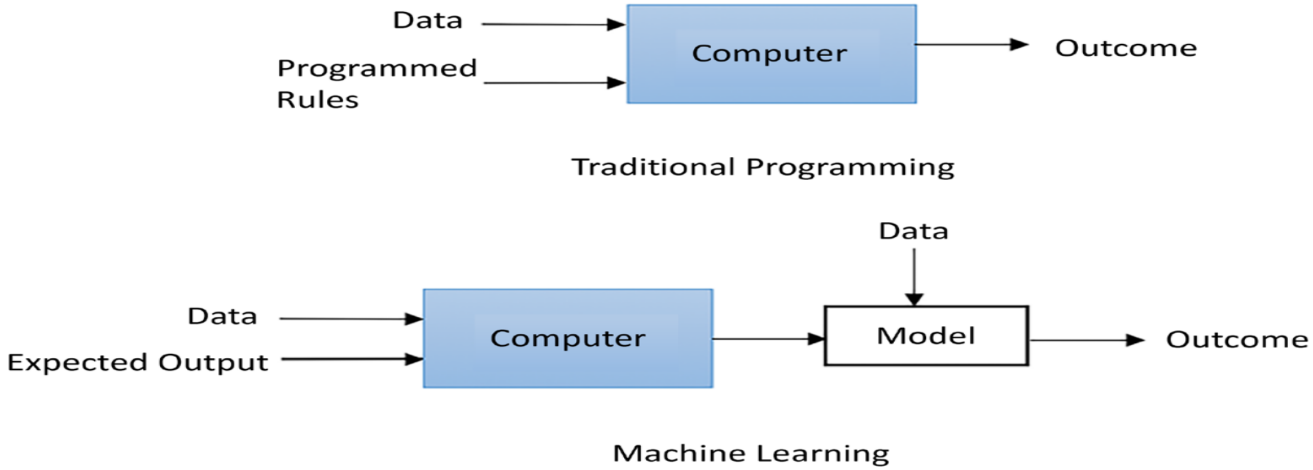


Figure: Machine learning versus traditional programming

- **Traditional Programming**: Computer follows a set of **predefined rules** to process input data and produce an outcome.
- **Machine Learning**: Computer **mimics human thinking**, interacts with data to derive patterns (mathematical models), and applies these to new data. It does **not receive explicit and instructive coding**.

Prerequisites

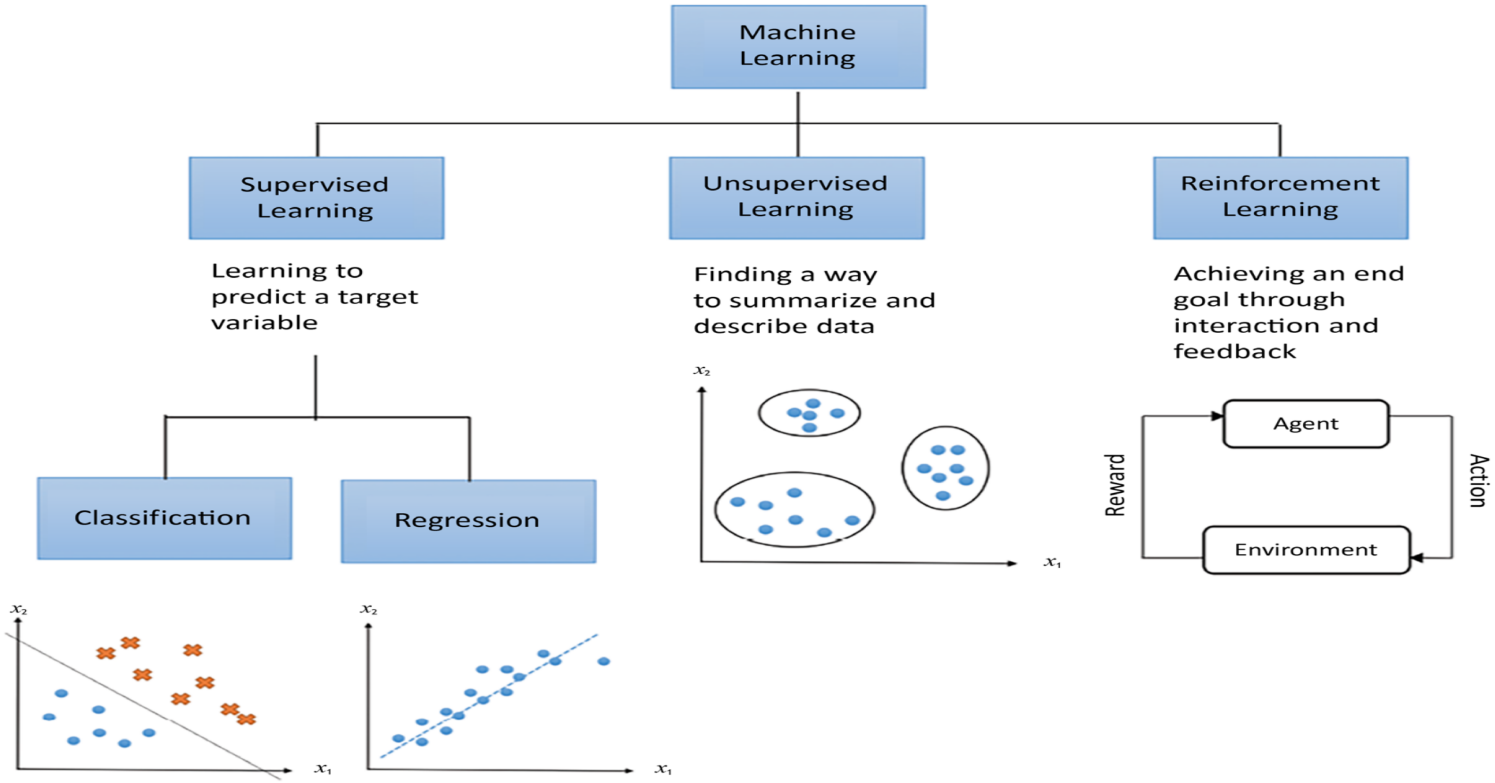
- **Prerequisites for this Course**
  - Prior knowledge of **Python coding** is assumed.
  - Basic familiarity with **statistical concepts** will be beneficial.
  - Some basic knowledge of **probability theory** and **linear algebra** helps understand mechanics, but deep mathematical details are avoided.

ML Evolution

- **A Brief History of ML Algorithm Development**
  - Machine learning algorithms continue to evolve rapidly.
  - **Deep Learning**: Became significant with the rise of **GPUs** (Graphic Processing Units), resembling how humans learn.
  - **Latest Developments**: Include transfer learning, generative models, and reinforcement learning, which are backbones of AIGC.
  - **Moore's Law**: Computer hardware improves exponentially, giving credibility to predictions of true machine intelligence.

Types of Machine Learning

- A machine learning system processes input data (numerical, textual, visual, audiovisual) and produces an output (e.g., floating-point number, integer category).
- **No Free Lunch Theorem**: No single algorithm is universally superior across all possible datasets and problem domains.



Types of Machine Learning

1. **Supervised Learning**
   - **Data**: Comes **with labels** (descriptions, targets, desired output).
   - **Goal**: Find a general rule mapping input to output, then use it to label new data.
   - **Subdivisions**:
     - **Regression**: Predicts **continuous-valued responses** (e.g., house prices, stock prices).
     - **Classification**: Attempts to find the appropriate **class label** (e.g., positive/negative sentiment, loan default).
2. **Unsupervised Learning**
   - **Data**: Only indicative signals, **without labels** (unlabeled data).
   - **Goal**: Find hidden structure, discover hidden information, or describe the data.
   - **Applications**: Anomaly detection (fraud), grouping customers (marketing campaigns), data visualization, dimensionality reduction.

Types of Machine Learning

3. **Semi-supervised Learning**
   - Uses a large amount of unlabeled data alongside a small amount of labeled data for training.
   - Applied when fully labeled datasets are expensive to acquire (e.g., hyperspectral remote sensing images).
4. **Reinforcement Learning**
   - Learning from experience by interacting with an environment to maximize a numerical reward.
5. **Self-supervised Learning**
   - Learns from unlabeled data by generating its own labels.

Generalizing with Data

- **Challenge**: Processing the diversity and noisiness of data.
- **Data Representation**: In Python, data is typically represented as numbers or text; text is transformed into numerical values.
- **Training and Validation Sets**:
  - Analogous to mock exams.
  - Help verify model performance in a simulated setting and fine-tune for greater accuracy.
- **Learning Rules**: Instead of explicit programming, ML allows the machine to consume data and figure out the underlying rules (e.g., tax rules, autonomous car navigation).
- **Measuring Success (Loss Function)**:
  - **Supervised Learning**: Compare results against expected values.
  - **Unsupervised Learning**: Measure success with metrics like how similar data points within a cluster are.
  - **Reinforcement Learning**: Program evaluates moves using a predefined function (e.g., in a chess game).

Overfitting, Underfitting, and the Bias-Variance Trade-off

- **Overfitting**
  - A model fits the existing observations **too well** but fails to predict future new observations.
  - Occurs when describing learning rules with too many parameters relative to small observations, or when the model is excessively complex.
  - **Analogy**: Memorizing answers for all questions instead of understanding the underlying concepts.

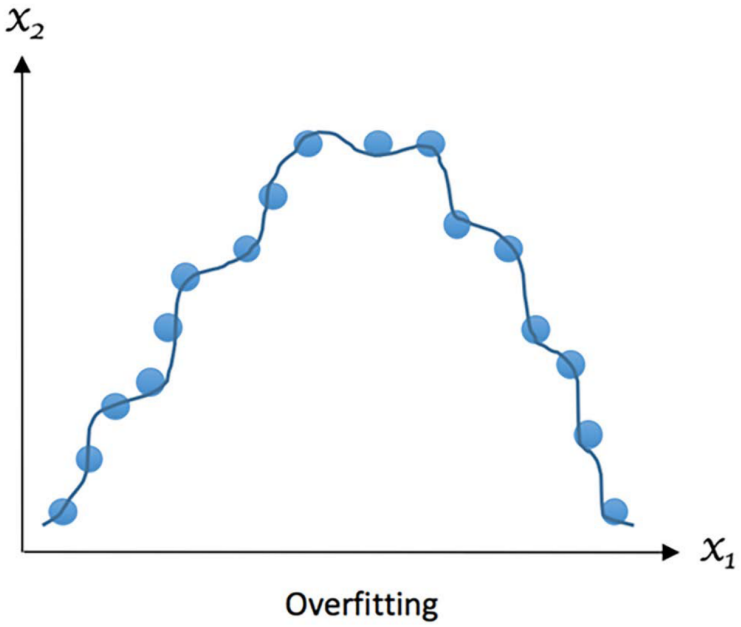


Figure: Example of overfitting

- **Underfitting**
  - The model **doesn't fit the data well enough** or capture enough of the underlying pattern.

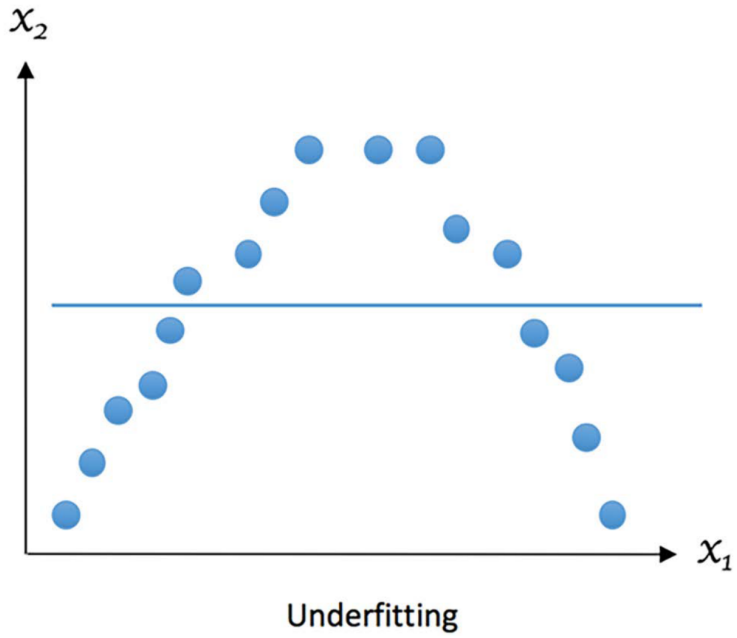
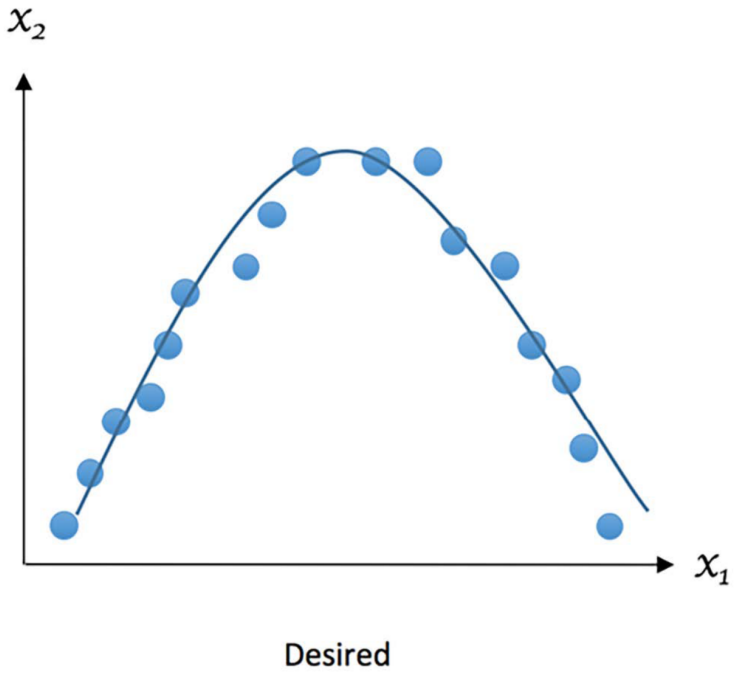


Figure: Example of underfitting

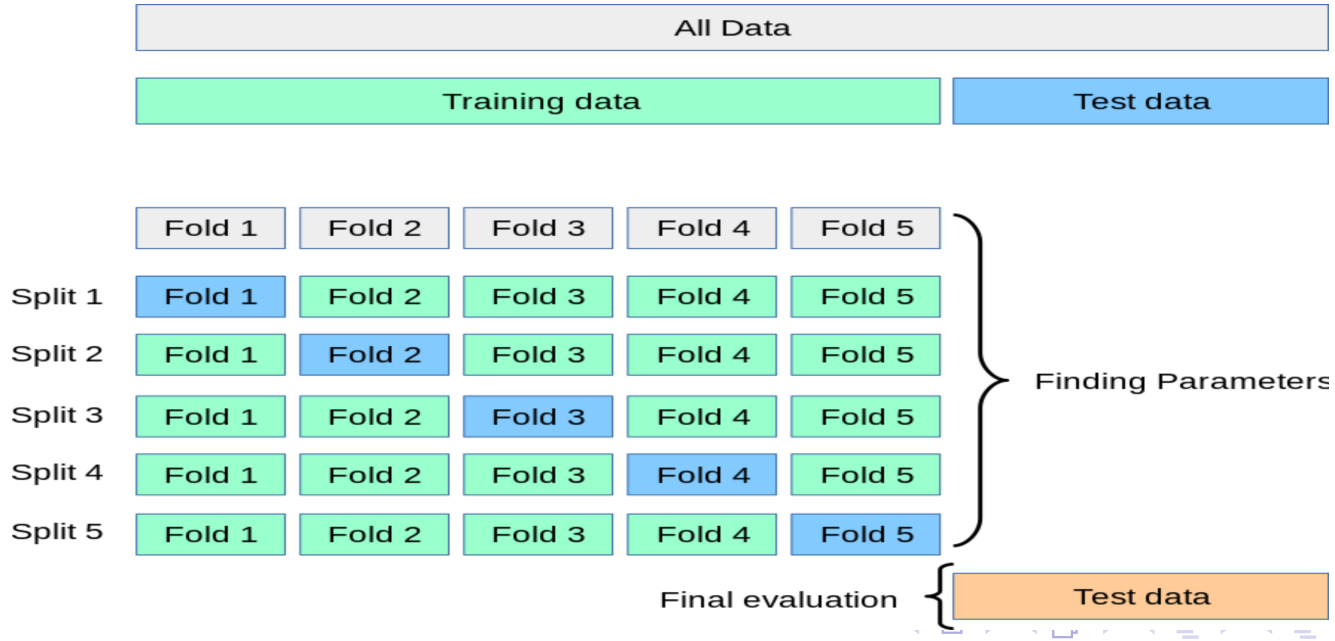
Bias-Variance Trade-off

- **Goal**: Avoid both overfitting and underfitting.
- **Bias**: Error stemming from incorrect assumptions in the learning algorithm. **High bias results in underfitting**.
- **Variance**: Measures how sensitive the model prediction is to variations in the datasets.
- **The Trade-off**: In practice, decreasing one (bias or variance) often increases the other.



Avoiding Overfitting with Cross-Validation

- Ensures the selected model generalizes well to unseen data.
- **Non-exhaustive Scheme: k-Fold Cross-Validation** (Most widely used)
  - Randomly split original data into **k equal-sized folds**.
  - In each trial, **one fold** becomes the **testing set**, and the rest are the **training set**.
  - Repeat this process **k times**, with each fold serving as the testing set once.
  - **Average the k sets of test results** for evaluation.



- **Stratified K-fold**: This method is similar to K-fold cross-validation but ensures that each fold contains approximately the same proportion of classes as the original dataset.
- **Exhaustive Scheme: Leave-One-Out-Cross-Validation (LOOCV)**
  - Each sample in the dataset is used as a testing set once.

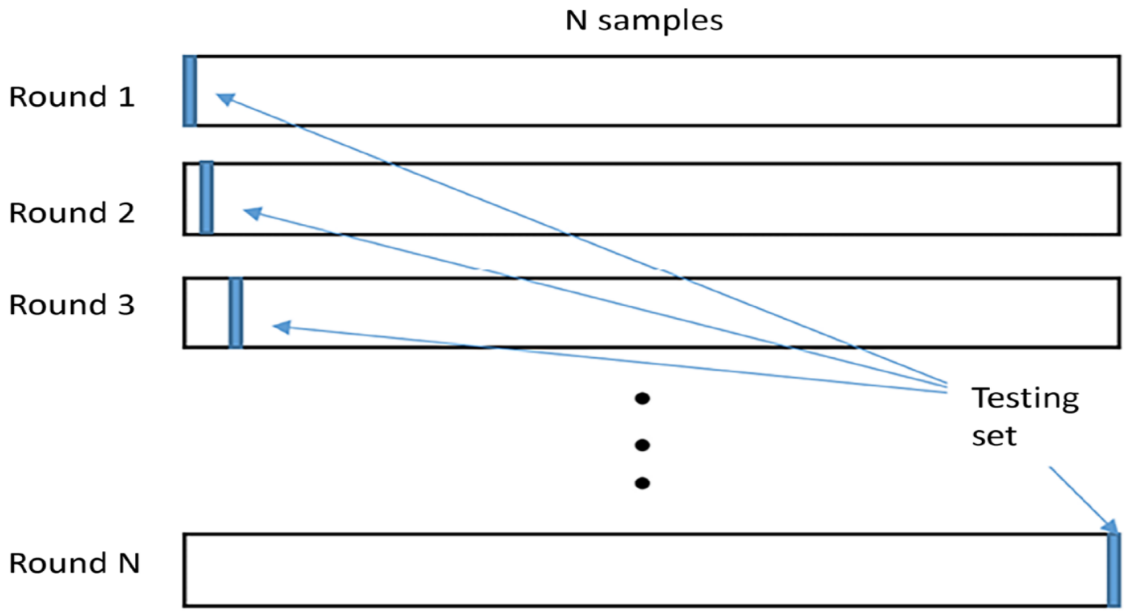
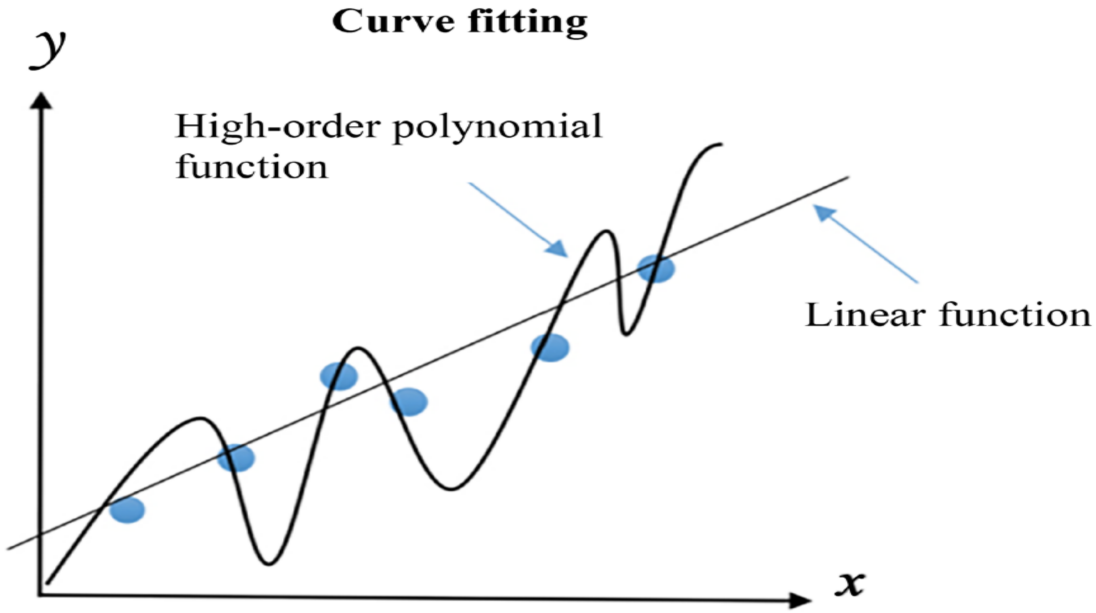


Figure: Workflow of leave-one-out-cross-validation

Avoiding Overfitting with Regularization

- **Regularization**: Techniques to reduce overfitting by **imposing penalties** on model complexity, discouraging excessively strict rules learned from training data.
- **Concept**: Reduces the influence of high orders of a polynomial.



- **Early Stopping**: Stop a training procedure early to produce a simpler model and control complexity, making overfitting less probable.
- **Optimal Level**: Regularization should be fine-tuned to an optimal level; too small has no impact, too large results in underfitting.

Avoiding Overfitting with Feature Selection and Dimensionality Reduction

- **Features**: In machine learning, each column in a data matrix represents a variable or "feature".
- **Dimensionality**: The number of features in the data.
- **Challenge**: Text and image data are often very **high-dimensional**, requiring specific approaches.
- **Techniques**:
  - **Feature Selection**: Choosing a subset of relevant features.
  - **Dimensionality Reduction (Feature Projection)**: Transforming high-dimensional data into a lower-dimensional space while retaining as much information as possible.
    - **Purpose**: Makes high-dimensional data easier to visualize and reduces redundancy from correlated features.
    - **Examples**: Principal Component Analysis (PCA) for linear transformation, t-SNE (t-distributed Stochastic Neighbor Embedding) and Non-negative Matrix Factorization (NMF) for non-linear transformation.

Data Preprocessing and Feature Engineering

- Plays a **crucial and foundational role** in machine learning, like laying the groundwork for a building.
- **Preprocessing and Exploration**
  - Becoming familiar with the data through scanning and visualization.
  - Ultimately, aim to represent data as a **grid of numbers**.
  - Identify **missing values**, **value distributions**, and **feature types** (binary, categorical, ordered categorical, quantitative).
- **Dealing with Missing Values**
  - Can occur due to inconvenience, expense, or inability to measure certain quantities.
  - Identified by scanning, counting values, or looking for encoded missing values (e.g., 999,999 or -1).

Data Preprocessing and Feature Engineering

- **Label Encoding**
  - Replaces each categorical label with an integer.
  - **Problem**: May imply an unintended order (e.g., Africa = 1, Asia = 2 implies Asia is "greater" than Africa), unless an order is naturally expected.

Label	**Encoded Label**
Africa	1
Asia	2
Europe	3
South America	4
North America	5
Other	6

Data Preprocessing and Feature Engineering

- **One-Hot Encoding (One-of-K)**
  - Uses **dummy binary variables** (0 or 1) for categorical features.
  - Avoids the problematic implied order of label encoding.
  - Requires k-1 or k dummy variables for k unique values.
  - Produces a **sparse matrix** (many zeros), handled well by scipy.

Continent	IsAfrica	IsAsia	IsEurope	IsSam	IsNam
Africa	1	0	0	0	0
Asia	0	1	0	0	0
Europe	0	0	1	0	0
South America	0	0	0	1	0
North America	0	0	0	0	1
Other	0	0	0	0	0

Data Preprocessing and Feature Engineering

- **Scalling**: Normalization, Standardization, Percentile split
- **Dense Embedding**
  - Provides a **compact, continuous representation** of categorical features.
  - Captures **semantic relationships** based on co-occurrence patterns.
  - **Example**:
    Africa: $[0.9, -0.2, 0.5]$
    Asia: $[-0.1, 0.8, 0.6]$
    Europe: $[0.6, 0.3, -0.7]$
    South America: $[0.5, 0.2, 0.1]$
    North America: $[0.4, 0.3, 0.2]$
    Other: $[-0.8, -0.5, 0.4]$
    South America and North America are closer together than those of Africa and Asia

- **Feature Engineering**: creating or improving features (not necessarily improve model performance)
- **Techniques**:
  - **Polynomial Transformation**: Creating new features from interactions between existing ones (e.g., $a^2 + ab + b^2$ from a and b).
  - **Binning**: Separating feature values into several bins (e.g., precipitation into "true/false" for rain, or ages into groups like "18 to 24").
    - Can lead to loss of information but may reduce overfitting and improve speed/memory.

Combining Models (Ensemble Learning)

- The idea is to **employ multiple models** to make better decisions than any single model alone.
- **Analogy**: Teamwork leads to better outcomes.
- **Techniques**:
  - **Voting and Averaging**:
  - **Bagging (Bootstrap Aggregating)**
  - **Boosting**
  - **Stacking**

Evaluation measures of a model

- **Regression**: MSE, RMSE, etc.
- **Classification**: Accuracy, Precision, Recall, F1-score, AUC

$$MSE = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

$$Accuracy = \frac{tp + tn}{tp + tn + fp + fn}$$

$$Precision = \frac{tp}{tp + fp}$$

$$Recall = \frac{tp}{tp + fn}$$

$$F1\text{-}score = \frac{2 * Recall * Precision}{Recall + Precision}$$

Popular python libraries

Tasks	**Python Libraries**
Data analysis	NumPy, SciPy, and pandas
Data visualization	Matplotlib, and Seaborn
Modeling	scikit-learn, TensorFlow, Keras, and PyTorch